# PRACTICAL - 03

**Aim:** Distinct points (Point.java + Driver.java)

## Code Screenshot:

Point.java

```java
import java.util.Objects;

public class Point {
    private int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public String toString() { return "(" + x + ", " + y + ")"; }

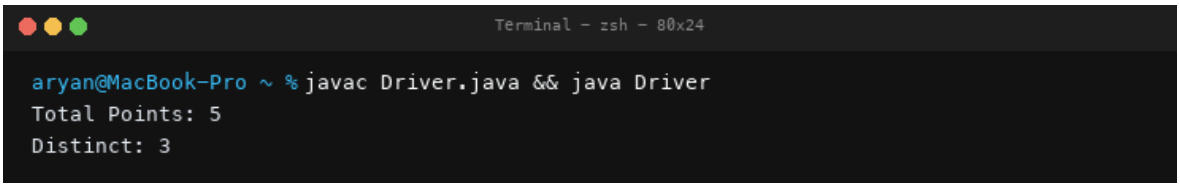
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    @Override
    public int hashCode() { return Objects.hash(x, y); }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Point[] points = {
            new Point(1, 2), new Point(3, 4),
            new Point(1, 2), new Point(5, 6), new Point(3, 4)
        };
        int distinctCount = 0;
        for (int i = 0; i < points.length; i++) {
            boolean duplicate = false;
            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) { duplicate = true; break; }
            }
            if (!duplicate) distinctCount++;
        }
        System.out.println("Total Points: " + points.length);
        System.out.println("Distinct: " + distinctCount);
    }
}
```

## Output Screenshot:




## Learning Outcome:

- Mastered the contract between equals() and hashCode() in Java.
- Overrode Object.toString() for human-readable coordinate representation.
- Implemented array deduplication logic using equality checking.
- Learned pattern matching for instanceof introduced in modern Java.
- Understood hashing consistency requirements when overriding equals.

**Aim:** Duplicate card check (Card.java + Driver.java)

## Code Screenshot:

Card.java

```java
import java.util.Objects;

public class Card {
    private String rank, suit;
    public Card(String rank, String suit) { this.rank = rank; this.suit = suit; }

    @Override
    public String toString() { return rank + " of " + suit; }

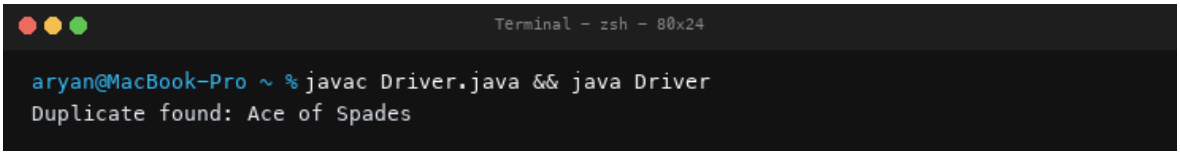
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Card c)) return false;
        return Objects.equals(rank, c.rank) && Objects.equals(suit, c.suit);
    }

    @Override
    public int hashCode() { return Objects.hash(rank, suit); }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Card[] cards = {
            new Card("Ace", "Spades"),
            new Card("King", "Hearts"),
            new Card("10", "Diamonds"),
            new Card("Ace", "Spades")
        };
        for (int i = 0; i < cards.length; i++) {
            for (int j = 0; j < i; j++) {
                if (cards[i].equals(cards[j])) {
                    System.out.println("Duplicate found: " + cards[i]);
                    return;
                }
            }
        }
    }
}
```

## Output Screenshot:




## Learning Outcome:

- Modeled compound domain entities with composite equality semantics.
- Applied string formatting in overridden toString() representations.
- Detected duplicates with early-exit search algorithms.
- Understood how Objects.equals() safely handles null references.
- Reinforced object identity vs structural equality concepts.